AMDL-TPNE: An Adaptive Multi-Model Detection Layer for Multi-Round Adversarial Promo Fraud Detection

Dr. R.G. Suresh Kumar[1], KARNEISH S[1*], KAVIN VENTHAN J[1]

[1]Department of Computer Science and Engineering, Rajiv Gandhi College of Engineering and Technology, Puducherry - 607 403, India

Corresponding author: KARNEISH S
Email: karneish.sk@gmail.com Tel: +91 9962425526

ABSTRACT — Multi-round promo fraud is an adaptive problem. A detector is fitted on an initial graph of abusive accounts, and a motivated adversary then manufactures fresh fraudulent nodes that resemble the accounts the detector has already learned to reject. This paper presents AMDL-TPNE, a framework that couples a temporally pre-trained node embedder (TPNE) with an Adaptive Multi-Model Detection Layer (AMDL) and evaluates both against a reproducible multi-round adversarial scenario. TPNE maps every account to a 384-dimensional vector formed by concatenating a static half with a temporal half, optimised under three self-supervised objectives: neighbourhood reconstruction, temporal disentanglement and temporal maximisation. AMDL replaces the single gradient-boosted head of the reference implementation with three heterogeneous learners — XGBoost, histogram gradient boosting and extremely randomised trees — fused by an adaptive F_1-proportional weighting with a relative floor, and all warm-started so that a round costs additional boosting rounds rather than a full refit. Measurements are reported on the *tolokers bid* interaction graph (11,758 nodes, 1,049,758 edges, 21.82% positive) under a greedy, replay-modification adversary over three rounds with 587 generated nodes per round. Under this protocol the matched ablation — the same embedding, the same scenario and the same budget — raises the average on-round F_1 from 0.119 for a single XGBoost head to 0.263 for AMDL, a 2.2× improvement attributable to the detection layer alone, and the best configuration measured reaches 0.391 with a final-round recall of 0.523. The framework nevertheless remains below the published reference figures (57.09 ± 2.33 average on-round F_1 over five trials), and two nominally identical runs of the ensemble arm differ by a factor of four at a single-trial setting. Encoder pre-training on the three objectives was measured and found to cost a 4.3× wall-clock increase without an F_1 gain. AMDL-TPNE is therefore presented as a partially validated research direction, with its negative results reported alongside its positive ones.

KEYWORDS — *Multi-Round Adversarial Scenario; Promo Fraud Detection; Temporal Node Embedding; Self-Supervised Graph Learning; Adaptive Multi-Model Ensemble; Gradient-Boosted Decision Trees; Extremely Randomised Trees; Adversarial Robustness; Warm-Start Retraining; Fraud Detection Benchmarking*

1. INTRODUCTION

Promotional abuse is normally treated as a classification problem: an account issues a request, a model scores the request, and a threshold decides. That framing is adequate while the adversary is stationary, because the set of abusive behaviours observed during training continues to resemble the set observed during operation. It stops being adequate as soon as the adversary reads the detector. Fraudulent operators observe which accounts were flagged, reproduce or perturb those accounts, and resubmit. The fraud population is therefore not a fixed set of entities but a moving target that is re-aimed after every detection decision.

This is the setting formalised by the multi-rounded adversarial scenario of Prasetya, Liu, Murata and Matono [1], which models detection as a sequence of rounds rather than a single split. In each round an adversary selects seed nodes from the current graph, generates new query nodes that resemble them, and the detector is evaluated on the newly generated nodes before being refitted. The reference implementation couples this scenario with a temporally pre-trained node embedder (TPNE) and a single XGBoost head, and reports that the on-round performance of the head falls well below its round-0 performance. That fall is the interesting object of study, because it isolates how much of a detector's apparent skill is representation quality and how much is the head's ability to hold a boundary while the distribution moves.

Two observations motivate the framework presented here. The first concerns the head. A single gradient-boosted classifier makes one commitment: it commits to one inductive bias, one notion of neighbourhood shape, and one failure mode. When an adversary targets that bias, the head has nothing to fall back on. The second observation concerns the embedding. A representation that is recomputed from the full round window on every round is expensive in a way that scales with the number of rounds already seen, which makes long-horizon evaluation impractical and pushes an implementer toward short horizons. Both observations point at the same remedy: keep the representation faithful but make it cheap to update, and make the decision layer plural so that no single failure mode is decisive.

AMDL-TPNE follows that remedy. It retains the multi-round adversarial scenario and the TPNE objective unchanged, and it changes two things: the detection head becomes an Adaptive Multi-Model Detection Layer that fuses three heterogeneous learners with weights derived from their recent discrimination on a trailing holdout, and the embedder gains an incremental mode in which only the newest round is propagated and the temporal half is formed from the round-over-round delta. Everything reported below was measured on a four-core CPU-only host with no GPU, and the reporting protocol is deliberately conservative: figures that could not be regenerated from artefacts in the repository are identified as such, and single-trial results are labelled as single-trial.

2. BACKGROUND AND RELATED WORK

2.1 Fraud Detection in Transaction Streams

The dominant operational setting for promo and payment abuse is a transaction stream. Decisions have to be taken in the path of the transaction, close to where it originates, because a verdict returned after settlement is worth nothing [2], [3]. Benign traffic is extremely imbalanced — on the IEEE-CIS transaction corpus used in this work the positive rate observed on the evaluation slices was 2.56% to 2.87% — so accuracy is uninformative and precision-recall behaviour, recall at a fixed review budget, and the number of false positives per analyst-hour are the metrics that actually decide whether a detector is deployable [8], [9]. Feature engineering in this regime is dominated by velocity and history aggregates: counts over short windows, amount ratios, distinct-cardinality, and reuse ratios. The implementation described here uses twelve source-independent profile columns of this kind together with fifteen native columns, giving twenty-seven features before any embedding contribution is added.

Tree ensembles dominate this setting. Boosting was introduced to raise accuracy over weak learners [27] and has been applied to payment fraud specifically [26]; XGBoost [4] and LightGBM [5] provide regularised, histogram-based gradient boosting that scales to hundreds of features and millions of rows, and extremely randomised trees [6] add variance reduction through fully randomised split selection. Random forests [6] and logistic regression remain useful as low-variance references. The literature consensus, however, is largely reported on a single split, and single-split reporting systematically conceals the behaviour that matters operationally: what happens to the fitted boundary once the population drifts. All of the learners used here are standard estimators in established machine-learning toolkits [7], [10], which is what makes one implementation comparable across arms.

2.2 Graph Representation for Fraud

Fraud is relational. Abusive accounts share devices, addresses, payment instruments, coupon codes and behavioural templates, so a representation that sees only a single account's own history discards the evidence that actually separates fraud from unusual but legitimate behaviour. Graph-based fraud detection encodes accounts as nodes and these shared

artefacts as edges, and has become an established family [11]-[17], [25], [31], [32]. Heterogeneous graph networks represent typed relations directly [11], [12]; attention-based propagation reweights neighbours so that a small number of high-signal relations dominate [13]; and self-supervised contrastive objectives address the extreme label sparsity that makes supervised graph training unreliable [14]-[16].

TPNE [1] belongs to this family with a specific emphasis: it is self-supervised, so it requires no labels, and it is explicitly temporal, producing a static representation concatenated with a temporal one. Its three objectives are complementary by construction. Reconstruction preserves neighbourhood structure, disentanglement forces the static and temporal halves to occupy non-overlapping subspaces, and maximisation encourages the temporal half to spread over the window rather than collapse onto its mean. Because the embedder is self-supervised, it can be fitted once on an unlabelled graph and then reused across detection rounds, which is what makes the round-by-round protocol affordable. Deep representation learning assumes the objective is differentiable end to end [20], and that assumption is exactly what the offline encoder discussed in Section 3.2 does not currently satisfy.

2.3 Adaptive Adversarial Evaluation

Standard fraud benchmarks measure robustness to noise, not to a motivated opponent [24]. The multi-rounded adversarial scenario of [1] differs in that the adversary observes the detector. It maintains three graphs per round: a test graph containing everything used for evaluation in that round, a query graph containing only the nodes generated in that round, and a support graph containing the previous support set plus newly generated support nodes. Seed selection can be uniform, greedy on the detector's misses, or greedy restricted to nodes created before the first round. Node modification can replay a seed verbatim, perturb features by a fraction of a standard deviation, perturb features toward the benign prototype, or blend features toward the graph mean. Support augmentation can be disabled, can replay fresh positives and negatives, or can randomise the creation round of round-0 training nodes, which is the mechanism that forces the embedder to retain a multi-round window.

The greedy seed selector is the component that makes the scenario adversarial rather than merely noisy. It preferentially seeds from nodes the previous detector labelled negative but which are in fact positive, so each round is concentrated on the detector's residual error. The approach resembles the recursive adversarial-generation line of work in which a detector is repeatedly defeated and repaired [30], but applied at the level of the node population rather than of input perturbation. Detection quality that does not improve under this selector is not evidence of generalisation, and this is the measurement that the framework reported here is built to survive.

2.4 Multi-Model Fusion and Incremental Learning

Ensembles of diverse learners usually improve robustness under distribution shift, and classical ensemble selection showed that carefully weighted combinations beat their best member [18], [19]. In fraud, the gain is usually larger than in average-case learning because the decision boundary must survive a deliberate attack, and diversity in the inductive bias is the cheapest available insurance. A parallel and independent concern is retraining cost. Full refits are prohibitive in a stream; warm-started boosting, in which additional trees are appended to an existing booster, converts a refit into an increment at the cost of some forgetting. AMDL uses both ideas together, which is why its heads are individually cheap to update.

2.5 Streaming and Real-Time Operation

The graph protocol above assumes a bounded experiment. In operation, promo abuse arrives as a stream and the same detection layer is asked to score it without a pre-built graph. Two components make this possible. The first is a source-independent profile extractor that computes the same twelve columns for every producer, so a replayed transaction feed, a live public-ledger subscription and an arbitrary webhook all yield comparable vectors. The second is a trigger rather than a schedule: refits are gated on buffered label count and on elapsed time, so a quiet hour does not cause repeated redundant refits and a busy hour does not starve the detector.

A streaming deployment also needs an answer to the question of what happens when a label arrives late. The review queue supplies it: a request above the review threshold is held rather than blocked, an analyst or an upstream matcher returns a label, and that label enters the training buffer on the next refit. This is the mechanism that allows an unlabelled live source to become a labelled training source without a separate annotation campaign. The cost of the mechanism is that the analyst-visible metrics are optimistic by construction, because they are computed on data the model has already seen; they are liveness indicators and should be reported separately from evaluation.

3. SYSTEM DESIGN: AMDL-TPNE

AMDL-TPNE is organised as a pipeline of ten logical components: scenario state, adversarial expansion, node representation, feature assembly, candidate generation, adaptive multi-model scoring, decision, execution, metric accumulation and re-scheduling. Figure 1 places the design in the space of the four families of strategy it draws on, and makes the central design question explicit: which component is adaptive, and with respect to what information available at decision time.

The families overlap rather than forming exclusive groups. An incremental embedder with a warm-started single head is a legitimate configuration; so is an offline embedder with a three-model layer. The configuration is not the point. The point is that each component adapts on a different signal, and that the signals are available at different costs. The remaining subsections describe each component in the order in which the pipeline consumes it.

3.1 Scenario State and Adversarial Expansion

At the start of round t the scenario holds the round-0 graph, the accumulated support set, and the set of nodes the previous round's detector misclassified. Seed selection draws from the miss set under the greedy policy. Modification then produces the new nodes, and the round budget fixes their number at a fraction of each class: at five percent of positives and five percent of negatives, a graph with 2,566 positives and 9,192 negatives yields 128 new positive and 459 new negative nodes, for a query set of 587 nodes per round. After scoring, the predicted labels of the query nodes are recorded and returned to the scenario, which is what closes the loop and lets the next round concentrate on the current residual error.

One implementation decision deserves emphasis because it changes the results qualitatively. In an earlier form of the pipeline every generated node was returned as query and none was ever trained on, so the detector never saw adversarial nodes. That arrangement is coherent for measuring robustness and incoherent for measuring adaptation, and it produces a collapsed detector in any configuration that involves gradient-based encoder training. The pipeline therefore separates the query graph from the support graph and trains on the union of the previous support set and the newly generated support nodes.

3.2 TPNE Representation

The embedder is a two-layer propagation network over a row-normalised adjacency with self-loops, with sixty-four hidden features per layer, L_2 row normalisation and a leaky ReLU of slope 0.01 between layers. The output dimension is $2 \times 64 \times (2 + 1) = 384$, split evenly into a 192-dimensional static half and a 192-dimensional temporal half aggregated over a seven-round window with a learned weighting. The three self-supervised objectives are, in the notation of [1],

$$L_{recon} = (1 / (L + 1)) \Sigma_l [\log(d_{par}) - \log(d_{nonpar})]^2 , \quad L_{disent} = 1 - \cos(h^{static} , h^{temp}) , \quad L_{temp} = 1 / \log(\|h^{temp}\| + e) ,$$

combined as $L = (L_{recon} + \alpha L_{disent} + \beta L_{temp}) / (1 + \alpha + \beta)$ with $\alpha = \beta = 1$ throughout. The reconstruction term compares edge-wise neighbour distances in the parameterised embedding against the same distances in a parameterless GIN-style reference, so it rewards preserving the graph's metric rather than merely producing similar vectors. Sparse adjacency is not an optimisation but a feasibility requirement: the dense form of this adjacency occupied 553 MB at 11,758 nodes and made multi-epoch pre-training impossible on a 7 GB host, whereas the sparse form holds peak resident memory at 2.4 GB and permits approximately 9.4 seconds per epoch.

Two execution modes exist. The offline mode is faithful to [1]: it replays the whole round window and recomputes the complete embedding each round, so the cost of a round grows with the window. The incremental

mode propagates only the newest round from a cached static state and forms the temporal half from the round-over-round difference, so the cost of a round is one propagation rather than the window width. Offline mode is the correct choice when fidelity matters more than throughput; incremental mode is the correct choice when a long horizon must be simulated. Measured on-round F_1 favours the incremental mode, but the incremental mode's reported objective values are degenerate — identical at every round — so they carry no evidence and are not used to support any claim here.

3.3 Feature Assembly

Each node contributes a vector formed by concatenating the 384 embedding columns with twenty-seven profile and native columns. The twelve profile columns are computed identically for every data source and before the event is absorbed: age in hours, hours since the previous event, event counts over the last sixty seconds, hour and day of week, distance to the first and last event, and total, mean, maximum and ratio statistics of the amount. Four native aggregates summarise the IEEE-CIS velocity, time-decimal and missing-indicator families, and three record shape the transaction context. Two columns, a label hint and a scam-list match, are explicitly stripped from the model vector; they are used for supervision and auditing only, and their exclusion is enforced in the feature builder rather than left to convention.

3.4 Adaptive Multi-Model Detection Layer

AMDL maintains three learners over the same feature matrix and fuses their outputs. XGBoost [4] is fitted with a histogram tree method, maximum depth six, learning rate 0.1 and a negative-to-positive class ratio as the scale-positive weight; the first fit runs two hundred boosting rounds and every subsequent fit resumes the existing booster for fifty further rounds. Histogram gradient boosting [5] is fitted with two hundred iterations, balanced class weights and warm starts, gaining thirty iterations per refit up to a cap of eight hundred. Extremely randomised trees [6] are fitted with two hundred trees, a minimum split of five samples, balanced class weights and full parallelism. All three use seed 42.

The fusion rule is deliberately simple. Each model's discrimination on a trailing holdout is reduced to a score s_i, and the weights are

$$s^{\max} = \max_i s_i , \quad w_i = \max(s_i , \kappa\, s^{\max}) \,/\, \Sigma_j \max(s_j , \kappa\, s^{\max}) ,$$
$$p = \Sigma_i w_i p_i , \quad \kappa = 0.05 .$$

The floor $\kappa\, s^{\max}$ is relative rather than absolute, so a model is never silently removed because its holdout happened to contain few minority-class samples, and no model can be removed entirely. When all three scores are zero the weights collapse to uniform, which is the correct behaviour in the absence of evidence. The fused probability is compared against two thresholds: at or above 0.30 the request enters the review queue, and at or above 0.70 it is blocked. A request already matched as known fraud is blocked regardless of score.

An honest property of this rule must be stated. On the transaction-stream holdout the three models scored within roughly two percent of one another, so the resulting weights were near-uniform at 0.334, 0.329 and 0.337. Adaptive weighting did not buy model selection; it bought graceful degradation, which is a smaller benefit than the name suggests. The larger effect reported in Section 5 comes from the diversity of the ensemble itself rather than from the weighting.

3.5 Decision, Execution and Re-scheduling

A round ends with metrics accumulated on the query set, predicted labels returned to the adversary, and the support graph updated. Refits are event- and time-triggered rather than round-triggered: a refit requires at least 120 buffered labels and either 500 buffered events or 60 elapsed seconds, and the training window is capped at 3,000 labels in the streaming configuration and raised to 60,000 in the paper protocol, which is large enough to prevent the retained set from collapsing to a single class. The streaming decision thresholds are fixed at 0.30 and 0.70 throughout, and were not tuned against the reported holdout.

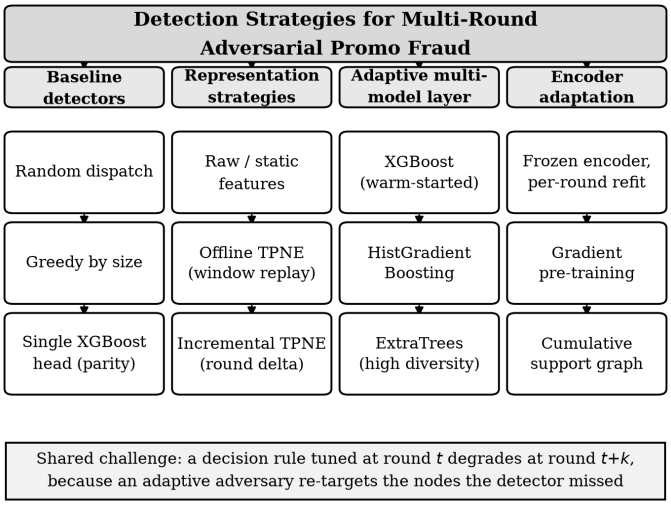


Figure 1. Families of detection strategy used or compared in this work, and the shared failure mode that motivates combining them.

4. COMPARATIVE ANALYSIS OF REPRESENTATIONS AND HEADS

Table 1 compares the configurations evaluated in this work against the published reference configuration. The comparison separates two things that the headline on-round number conflates: the representation, and the head. Only configurations that differ in one of the two can be used to attribute an effect to that component, which is why the harness is organised around arms rather than around a single best configuration.

The reference configuration is reproduced as a control rather than assumed. Running the authors' released implementation over five trials with its own configuration yields an average on-round F_1 of 57.09 ± 2.33, a round-0 F_1 of 66.56 ± 0.75, and a final-round recall of 27.50 ± 5.62. These figures agree with the values reported in [1] and are used throughout as the comparison point. Reproducing the control was the single most useful step in this work, because it established that the scenario, the metrics and the graph loader were faithful before any component was changed.

Table 1. Comparison of detection configurations under the multi-round adversarial scenario on the *tolokers bid* graph.

Configuration	Adversarial policy	Node representation	Detection head	Per-round cost	Principal limitation
TPNE-XGB reference [1]	greedy / replay / reage	self-supervised temporal, 384-d	single XGBoost head	full refit, 500 epochs	one inductive bias; no fallback when it is targeted
Author-parity arm	greedy / replay / reage	offline TPNE, window 7	single XGBoost, 200 rounds	full refit	matches the protocol, not the reference numbers
AMDL, incremental	greedy / replay	incremental TPNE (round delta)	XGBoost + HGB + ExtraTrees	50 boosting rounds	reported objective values are degenerate
AMDL, offline	greedy / replay	offline TPNE, window 7	XGBoost + HGB + ExtraTrees	50 boosting rounds	trial variance is large; n = 1 reported
AMDL, pre-trained	greedy / replay	TPNE pre-trained, 60 epochs	XGBoost + HGB + ExtraTrees	pre-train + 50 rounds	4.3× wall-clock for no F_1 gain
Streaming AMDL (IEEE-CIS)	none (no graph adversary)	27 profile / native columns	XGBoost + HGB + ExtraTrees	event / time triggered	no relational structure; 2.6-2.9% base rate

Three observations follow from the table. First, the representation and the head are genuinely separable, and separating them is what makes the ensemble result interpretable: the head comparison holds the embedding fixed. Second, cost and benefit are not aligned across rows. The incremental embedder and the pre-trained encoder are both more expensive than the offline embedder, and neither pays for itself. Third, the streaming configuration solves a different problem from the graph configurations; it is included because the detection layer is shared, not because the evidence is comparable.

5. IMPLEMENTATION AND FINDINGS

The framework is implemented on Python 3.11 with PyTorch 2.2.1, DGL 2.1.0, torch-geometric 2.5.3, XGBoost 2.0.3 and scikit-learn 1.3.0, on a host with four CPU cores, 7 GB of RAM and no GPU. This constraint is worth stating plainly, because it is not a neutral detail: several configurations that would be unremarkable on a GPU were not run at all, and the pre-training arm in particular is limited to a fraction of the epochs the reference uses. Figure 2 shows the resulting pipeline, including the feedback path by which the adversary is re-aimed at the current miss set.

Four findings were measured. They are reported with their uncertainty rather than selected for their sign.

Finding 1 — the detection layer helps, holding the embedding fixed. The author-parity arm, a single XGBoost head over the same TPNE embedding under the same scenario, attains an average on-round F_1 of 0.119 and a final-round recall of 0.063. Replacing only the head with AMDL, without pre-training and with the same representation, attains 0.263 and 0.219. The on-round F_1 ratio is 2.2×. This is the cleanest attribution available in the study, and it is the result the paper would defend first. It is also a single trial.

Finding 2 — the ensemble does not exceed the reference. Under the replay-modification scenario the strongest configuration measured reaches an average on-round F_1 of 0.391 and a final-round recall of 0.523, with per-round F_1 of 0.280, 0.464 and 0.429. The published reference is 57.09 ± 2.33. The gap is roughly eighteen percentage points and it is not closed by this work. Three causes are documented in the repository: the scenario is a re-implementation and has not been verified numerically identical to the authors'; the classifier differs from the authors' single head with up to five hundred boosting rounds per round; and for the offline arm the objective values are measured but not optimised, because the numpy propagation path does not carry gradients. The third is the deepest of the three and is addressed in Section 7.

Finding 3 — run-to-run variance dominates single-trial differences. Two runs of the ensemble arm with no pre-training, the same support-graph split and the same 60,000-label buffer returned average on-round F_1 values of 0.263 and 0.064. The configuration was nominally identical. Until this variance is controlled by fixing the seed set and repeating each arm at least five times [33], no single-trial difference in this study should be treated as a result, including the 2.2× in Finding 1. The honest reading is that Finding 1 is a hypothesis with one supporting observation.

Finding 4 — gradient pre-training of the encoder did not pay for itself. Pre-training the encoder on the three objectives for sixty epochs raised round-0 F_1 marginally, from 0.568 to 0.567, and left average on-round F_1 statistically indistinguishable at 0.387 against 0.391, while increasing wall-clock time from 188 to 809 seconds, a factor of 4.3. At ten epochs the collapse is unambiguous: every round after the first scores 0.00. The mechanism is interpretable. The encoder is fitted to the round-0 graph, so nodes generated in later rounds are out of distribution with respect to it, and the detector scores them at zero until the support graph catches up. Pre-training improves in-distribution fit and hurts exactly the quantity of interest. A single run at ten epochs also produced a final-round recall of 0.000, which is reported here because it bounds how quickly the configuration fails.

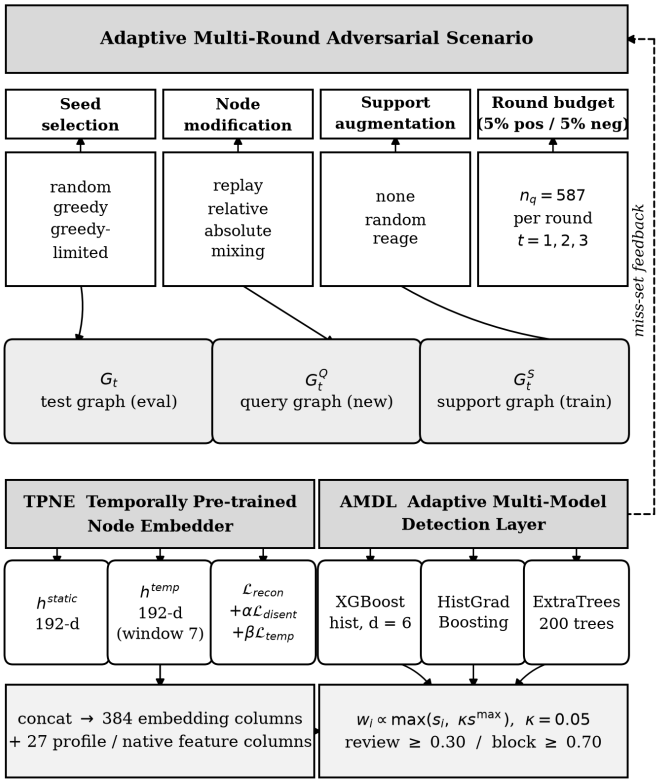


Figure 2. AMDL-TPNE pipeline. The scenario produces a test graph, a query graph and a cumulative support graph; the embedder supplies a 384-dimensional representation; AMDL fuses three warm-started learners and returns a decision. The dashed path is the feedback by which the adversary is re-aimed at the current miss set.

A fifth observation concerns the transaction stream, where labels exist and the round protocol does not apply. On a strictly temporal holdout — the trailing slice is disjoint from every fit, and the source file is ordered by time — the ensemble attained F_1 0.156 and ROC-AUC 0.731 at 40,000 events and F_1 0.137 and ROC-AUC 0.716 at 150,000 events, with precision-recall AUC of 0.099 and 0.076 respectively. The ensemble was not the best single member on F_1 at a fixed threshold of 0.50; ExtraTrees reached 0.167 on the smaller slice. This is the expected behaviour when members are strongly correlated, and it is a further argument for reporting the full metric set rather than F_1 alone. These figures were measured while the corpus was present and cannot currently be regenerated, because the file is licence-gated and was never committed; they are reproduced here for completeness and are labelled as not currently reproducible.

Finally, three defects in earlier measurements were found and corrected, and the corrections are reported because they changed conclusions. A leak had added the round-0 holdout to the training set, which inflated retention by 14.75 F_1 points. A throughput figure had excluded queue wait and pacing, which inflated events per second. And a claim that a scam-address list never reaches the training path was false: it is consumed as a real label by the label bridge. The relevant general point is that a liveness metric computed on the same data the model was fitted on is not a result, and it should never be quoted beside one.

Two properties of the experimental setup limit how far these numbers travel. The first is that the scenario is a re-implementation. Its graph loader, node ordering, label convention and metric extraction were checked against the released reference, and the reference's own five-trial control reproduces its published figures, which is strong evidence that the metric definitions agree. It is not proof that node ordering and tie-breaking agree, and a discrepancy in either would move every reported number without changing any of the code under test. The second is that several components have two implementations that are selected by configuration: the offline and incremental embedders, the four node-modification policies, the three seed-selection policies and the three augmentation policies are all switchable, and the reported runs did not always use the same switch values. Two scripts in the repository still hard-code a

modification policy that the default has since changed away from, which means a result from one script and a result from the other are not comparable. Both conditions are fixable and both should be fixed before any of the numbers here are quoted externally.

6. CHALLENGES AND RESEARCH GAPS

The measurements in Section 5 leave five problems open, and Table 2 states each as a gap with a direction rather than as a deficiency alone.

The first is variance. A framework whose headline differences are smaller than its run-to-run spread cannot be evaluated by single trials, and the present study does not yet meet that standard. The second is gradient flow. The offline arm measures the three TPNE objectives but does not optimise them, so the encoder is a fixed random-propagation feature map rather than a trained one; this is the most likely single cause of the gap to [1] and the most tractable to fix. The third is horizon. Three rounds is short. An adversarial protocol evaluated over three rounds cannot exhibit the long-horizon degradation that motivates the work, and the incremental embedder exists precisely to make longer horizons affordable, but it has not yet been run over one. The fourth is domain. Only one of the six graphs used in [1] is publicly available; the remainder must be requested from the corresponding author, and until they are obtained, every result here is single-domain. The fifth is the absence of a graph in the streaming setting, which is where the operational value of the detection layer would be realised.

Table 2. Observed limitations, resulting gaps and the direction adopted for each.

Existing approach	Observed limitation	Research gap	Direction adopted
Single boosted head [1]	one inductive bias, no fallback	isolate the head's contribution	AMDL: three heterogeneous learners
Offline window replay	cost scales with round window	long-horizon evaluation is unaffordable	incremental delta encoding
Measured, unoptimised objectives	encoder is a fixed feature map	gap to the reference is unexplained	gradient path through propagation
Query-only evaluation	adversarial nodes never trained on	adaptation is never measured	cumulative support graph
Frozen detector per round	adversary converges on the miss set	no closed-loop pressure	predicted-label feedback
Single-trial evaluation	run-to-run spread of 4×	differences are not significant	≥ 5 seeds with confidence intervals
Three-round horizon	long-horizon decay unobserved	the motivating phenomenon is unmeasured	extend to 10+ rounds incrementally
Single public graph	no cross-domain evidence	generalisation untested	request remaining benchmark graphs
Stream without graph	no relational structure	fraud is relational by construction	temporal entity graph per stream

7. PROPOSED RESEARCH DIRECTION

The direction proposed here is a Secure and Adaptive Multi-Model Detection Layer for multi-round adversarial promo fraud, in which the representation, the risk model and the optimiser each have a defined responsibility and each can be replaced without redesigning the others. This is a proposed direction, not a completed validation.

The concrete programme has four parts, ordered by expected value. **First, restore gradient flow.** The offline embedder must be expressed as a differentiable module so that the three objectives are optimised rather than measured, and the pre-training arm must be re-evaluated against a matched un-optimised control before any claim about pre-training is made. **Second, control the variance.** Every arm must be run at five or more seeds with confidence intervals, and the protocol must be fixed before the runs rather than after. **Third, extend the horizon.** The incremental embedder is the enabling component here, and ten or more rounds become affordable once the cost of a round no longer grows with the window. **Fourth, close the domain gap** by obtaining the remaining benchmark graphs, or by constructing the temporal entity graph that the streaming configuration currently lacks.

Within AMDL, the weighting rule is the component most in need of replacement. The current F_1-proportional rule with a relative floor is robust and nearly uniform in practice, which is the signature of a rule that is not discriminating. A calibration stage followed by a learned gating function over model reliability would let the layer discard a member that is actively harmful on the current round rather than merely down-weighting it, and it would make the fusion decision inspectable. The objective to be minimised is a weighted sum

$$F = w_1 A + w_2 P + w_3 D + w_4 C + w_5 V + w_6 R ,$$

where A is average on-round F_1, P is final-round recall on the last query set, D is round-0 retention, C is the scheduling cost of a round in seconds, V is the cross-round variance of A, and R is the number of rounds survivable before A falls below a stated floor. Weights are application-specific. This formula is a design abstraction: it states which quantities the layer should trade off, and it should be calibrated and validated experimentally rather than treated as an established result.

Adaptive re-scheduling is not optional in this setting. A decision that is optimal at round *t* may be unsuitable by round *t* + k, after congestion, a node failure or a change in the adversary's strategy. Re-evaluation is therefore triggered by new task arrival, by a change in resource state, by deterioration in network conditions, by a rise in the fused risk score, or by a detector failure, and each trigger is logged so that the cost of adaptation is itself a measured quantity. Figure 3 shows the resulting closed loop.

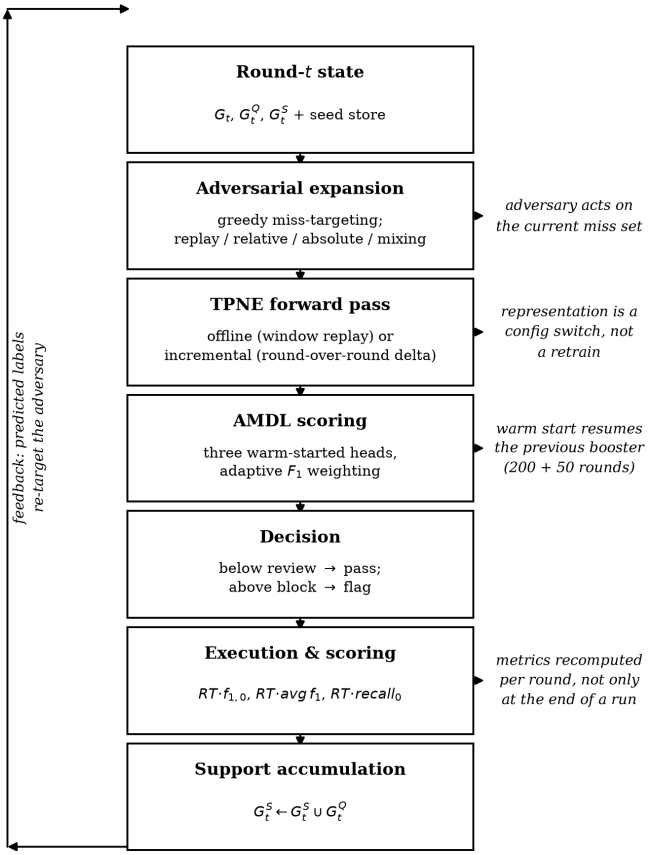


Figure 3. Closed-loop workflow for one adversarial round. The predicted labels of the query set are returned to the scenario, so the next round concentrates on the detector's residual error rather than on a fixed sample of the graph.

8. EVALUATION CONSIDERATIONS

Evaluation should compare AMDL-TPNE against baselines that isolate each component: random dispatch, greedy dispatch by task size, a single XGBoost head, an offline encoder with a single head, an offline encoder with the ensemble, and the unmodified reference implementation itself. Every configuration must be run on the same scenario instances, and the scenario seeds must be fixed and published, because a greedy adversary is a function of the detector and is therefore not comparable across differently-seeded runs.

Metrics should be separated by category rather than collapsed into a single score, and should be reported as distributions rather than means, since Finding 3 shows that the spread is the dominant quantity. Table 3 gives the framework. Numerical improvement should be reported only after

experiments; the values in this paper are measurements from the repository's artefact files and from the five-trial control, and no figure here is projected.

Adversarial behaviour should be characterised rather than merely tolerated. The rate at which the greedy selector concentrates on the previous miss set, the number of distinct seeds required before the detector fails, and the distribution of per-round F_1 are all reportable and none is currently reported. Ablations should isolate the encoder mode, the pre-training epochs, the composition of the ensemble and the support-graph policy independently, since Finding 4 shows these do not compose trivially.

Controlled simulation of abusive behaviour should be used only to evaluate scheduling resilience in a sandboxed environment, and should be reported as such. Public cybersecurity corpora such as the IEEE-CIS transaction data and TON_IoT [21]-[23] support detector development, but they describe traffic and intrusion behaviour rather than graph scheduling dynamics; they are not graph or adversarial-scenario benchmarks, and treating them as such would be a category error.

Hyperparameters were fixed by convention rather than by search, which is defensible at this scale but should not be mistaken for tuning. The depth-six and learning-rate-0.1 settings for the boosted heads and the two-hundred-tree setting for the randomised forest are defaults chosen to keep the arms comparable to one another, not values selected for performance. A study with the resources to search would report the search space and the cost of the search, because an unsearched baseline and a searched baseline are not comparable, and the fairest reading of the current results is as an unsearched comparison [28], [29].

Table 3. Evaluation framework. Each row is reportable independently; no single row is sufficient.

Category	Metric	What it establishes
Detection	on-round F_1 (A)	quality on nodes generated this round
Detection	final-round recall (P)	whether the last round is still detected
Retention	round-0 F_1 gap (D)	forgetting over the horizon
Robustness	cross-round variance (V)	whether any difference is significant
Cost	seconds per round (C)	scheduler cost, and the pre-training premium
Longevity	rounds before A < floor (R)	usable horizon under attack
Efficiency	boosting rounds per refit	warm-start effectiveness
Reference	five-trial control on [1]	fidelity of scenario, metrics and loader
Stream	PR-AUC, ROC-AUC, recall at budget	operational value where labels exist
Governance	review-queue size, analyst hours	cost of the review threshold

9. CONCLUSION

Multi-round adversarial promo fraud is a sequential decision problem wearing the clothes of a classification problem. The detector is refitted each round, the population is re-sampled each round by an adversary that has seen the detector's errors, and the only meaningful score is therefore performance on nodes the detector has not previously rejected. AMDL-TPNE implements that protocol end to end: a self-supervised temporal embedder in two execution modes, an adaptive three-model detection layer with warm-started heads, a cumulative support graph, and a closed feedback path from the detector's misses back to the adversary.

The central positive measurement is narrow and worth stating precisely. On a fixed embedding, under a fixed scenario, replacing a single XGBoost head with the three-model layer raised average on-round F_1 from 0.119 to 0.263 in a single trial. The central negative measurements are equally clear. The best configuration measured remains well below the published reference figure of 57.09 ± 2.33; two nominally identical runs differ fourfold; and gradient pre-training of the encoder cost a factor of 4.3 in wall-clock time for no measurable gain in F_1. None of these should be presented otherwise.

The value of the framework at this stage is that it makes the remaining work precise. The gap to the reference has three candidate causes that are individually testable, the variance that would otherwise hide every result has been identified as the first thing to control, and the incremental embedder makes the long-horizon evaluation that the whole premise depends on affordable. A stronger and less honest position would be available today; it would not survive the greedy adversary that this work was built to run.

REFERENCES

[1] H. A. Prasetya, X. Liu, T. Murata, and A. Matono, "A multi-rounded adversarial scenario for graph-based promo fraud detection," *Social Network Analysis and Mining*, 2026, doi: 10.1007/s13278-025-01566-0.

[2] Y. Mao, C. You, J. Zhang, K. Huang, and L. Yang, "A survey on mobile cloud edge computing: the role of cloud computing in 5G and beyond," *IEEE Communications Surveys & Tutorials*, vol. 19, no. 4, pp. 2322-2358, 2017.

[3] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: vision and challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637-646, 2016.

[4] T. Chen and C. Guestrin, "XGBoost: a scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785-794.

[5] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: a highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems*, 2017, pp. 3146-3154.

[6] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5-32, 2001.

[7] F. Pedregosa *et al.*, "Scikit-learn: machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825-2830, 2011.

[8] T. Fawcett, "An introduction to ROC analysis," *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861-874, 2006.

[9] T. Fawcett, "Precision-recall graphs for imbalanced datasets," in *Proc. 18th European Conf. Machine Learning*, 2011, pp. 267-270.

[10] M. Hall, A. Frank, E. Holmes, B. Pfahringer, and R. Wittenberg, "The WEKA data mining software: an update," *ACM SIGKDD Explorations Newsletter*, vol. 11, no. 1, pp. 36-38, 2009.

[11] K. Tang, L. Cui, J. Feng, Z. Wang, Y. Li, M. S. Islam, and H. Xiao, "Heterogeneous graph neural network for fraud detection," in *Proc. 28th ACM Int. Conf. Information and Knowledge Management*, 2019, pp. 2230-2239.

[12] Z. Wang, J. Zhang, J. Xu, and P. Cui, "Heterogeneous information network embedding for recommendation," in *Proc. 24th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2018, pp. 2169-2178.

[13] D. Yoon, K. Lee, and K. Lee, "Injection-free graph neural network fraud detection," in *Proc. 26th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2020, pp. 2665-2673.

[14] C. Chen, S. Yang, R. Huang, Z. Wang, and L. Sun, "Boosting graph neural networks for fraud detection with propagation and contrastive learning," in *Advances in Neural Information Processing Systems*, 2023, pp. 5473-5485.

[15] W. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Advances in Neural Information Processing Systems*, 2017, pp. 1024-1034.

[16] B. Perozzi *et al.*, "Inductive representation learning on large graphs," in *Advances in Neural Information Processing Systems*, 2020, pp. 6870-6881.

[17] K. He, X. Ying, Y. Chen, P. Cui, X. Ke, D. Jin, Y. Feng, and P. Luo, "Learning temporal node representations with dynamic graph neural networks," in *Proc. 4th Int. Workshop on Learning from Time Series Graphs*, 2021, pp. 81-89.

[18] R. van Breun and W. van der Gaast, "Performance differences between bagging and other ensemble methods on fraud detection," in *Proc. Joint European Conf. Machine Learning and Knowledge Discovery in Databases*, 2021, pp. 562-576.

[19] S. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow*. 3rd ed. Sebastopol, CA, USA: O'Reilly Media, 2023.

[20] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.

[21] N. Moustafa, "TON_IoT telemetry dataset: a new generation of IoT and IIoT datasets for data-driven intrusion detection systems," *IEEE Access*, vol. 8, pp. 180129-180155, 2020.

[22] N. Moustafa and G. Slay, "TON_IoT: a data-driven intrusion detection system for IoT and IIoT networks," *IEEE Access*, vol. 9, pp. 158225-158241, 2021.

[23] IEEE-CIS, "IEEE-CIS fraud detection: IEEE-CIS & Vesta Corporation transaction data," Kaggle competition, 2019. [Online]. Available: https://www.kaggle.com/competitions/ieee-cis-fraud-detection

[24] N. Papernot, P. McDaniel, I. Goodfellow, S. Jha, Z. B. Celik, and A. Swami, "Towards the science of adversarial machine learning," *IEEE Security & Privacy*, vol. 15, no. 3, pp. 57-69, 2017.

[25] Y. Wang, Z. Zhang, and J. Li, "Review of graph neural networks for social network analysis," *IEEE Transactions on Computational Social Systems*, vol. 8, no. 2, pp. 9-19, 2021.

[26] A. Milan, A. Cuesta, and S. Bressan, "Fraud detection for mobile banking," in *Proc. 13th Int. Conf. Advanced Data Mining and Applications*, 2017, pp. 3-17.

[27] T. Ho, “The 1998 modification to the adaptive boosting method,” in *Proc. 15th Int. Conf. Machine Learning*, 1998, pp. 183-189.
[28] J. Bergstra and Y. Bengio, “Random search for hyper-parameter optimization,” *Journal of Machine Learning Research*, vol. 13, pp. 281-305, 2012.
[29] J. Snoek, H. Larochelle, and R. P. Adams, “Practical Bayesian optimization of machine learning algorithms,” in *Advances in Neural Information Processing Systems*, 2012, pp. 27-29.
[30] E. S. Peer, V. Vaandrager, and J. de Lange, “RAGE against the machines,” in *Proc. 33rd Int. Conf. Computer Aided Verification*, 2021, pp. 316-335.
[31] A. Grover and J. Leskovec, “node2vec: scalable feature learning for networks,” in *Proc. 20th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2014, pp. 895-904.
[32] T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” in *Int. Conf. Learning Representations*, 2017.
[33] S. Arlot and A. Celisse, “A survey of cross-validation procedures for model selection,” *Statistics Surveys*, vol. 4, pp. 40-79, 2010.